

04 — Immagini che non arrivano: alt, error, SVG inline

Cosa succede davvero quando un `<img>` non si carica, perché il testo alternativo finisce a schermo, e come si costruisce un segnaposto che sembri una scelta e non un guasto. Esempi: `src/ui/segnaposto.js`, `src/ui/griglia-collezione/`, `src/ui/visore-carta/`.

1. Il problema, in concreto

Nei dati italiani di TCGdex parecchie carte hanno `immagine: null`: i set più vecchi (Set Base, Team Rocket...) non hanno nessuna scansione, e le Energie base generiche non appartengono a nessun set, quindi una scansione non può esistere per definizione. Sono decine di carte per collezione, non un caso limite.

Prima c'erano due modi di dirlo, ed erano entrambi sbagliati:

1. un **punto interrogativo** dentro un riquadro, che si legge come "l'app non sa cosa mostrarti" — cioè come un errore, non come una carta senza foto;
 2. per le immagini che *esistono* ma non arrivano (offline, o URL cambiato a monte), il **testo alternativo** stampato a caratteri di sistema in mezzo alla griglia, accanto all'icona di immagine rotta del browser.
-

2. alt non è una didascalia

`alt` è il testo che **sostituisce** l'immagine: lo annuncia uno screen reader, e il browser lo disegna a schermo quando l'immagine non c'è. Le due cose sono la stessa cosa, ed è il punto che si dimentica.

La regola sta nelle specifiche HTML: `alt=""` (vuoto, non assente!) dichiara l'immagine **decorativa**. Gli screen reader la saltano del tutto e il browser non ha niente da disegnare al suo posto.

Quindi la domanda non è "come descrivo l'immagine" ma **"se questa immagine sparisce, l'informazione si perde?"**:

Dove	Cosa c'è intorno	Scelta
Griglia della collezione	Il nome della carta è scritto sotto la miniatura	<code>alt=""</code>
<code><scheda-carta></code>	Il nome è nel titolo accanto	<code>alt=""</code>
Carosello di un mazzo	Il nome è nel <code>title</code> del pulsante e nella lista	<code>alt=""</code>
<code><visore-carta></code>	Niente: la scansione è il contenuto	<code>alt="Carta Pikachu"</code>

Un `alt` mancante del tutto (`<img src=...>` senza attributo) è il caso peggiore dei tre: gli screen reader, non sapendo che fare, leggono il nome del file.

Differenza con Angular: nessuna, ed è proprio questo il punto. Non è una questione di framework ma di piattaforma — la stessa regola vale in un template Angular, in JSX o in una stringa costruita a mano.

3. error: l'unico evento che ti avverte

Un'immagine che risponde 404 non lancia eccezioni e non rifiuta nessuna Promise. Emette un evento `error` sull'elemento, e basta. Se nessuno lo ascolta, resta a schermo la resa di default del browser.

```
img.addEventListener(
  'error',
  () => { img.outerHTML = segnaposto(carta, classe); },
  { once: true },
);
```

Tre dettagli non ovvi:

- **error non fa “bubbling”** come gli eventi normali: non lo si può intercettare con un solo gestore sul contenitore, come si fa con `click` in questo progetto. Va agganciato al singolo `<img>` (oppure catturato in fase di *capture*, che qui sarebbe più oscuro che utile).
- `{ once: true }` toglie il gestore dopo la prima chiamata: l'elemento sparisce comunque un istante dopo, ma è l'abitudine giusta — un gestore che si smonta da solo non tiene in vita l'oggetto a cui si riferisce.
- `outerHTML` sostituisce l'elemento *compreso se stesso*, che è esattamente ciò che serve: l'`<img>` non deve restare nel DOM, nemmeno nascosta.

Nel visore la sostituzione non ha senso (la cornice è grande quanto lo schermo): lì il gestore nasconde l'immagine e scrive nome e numero nella cornice, cioè riusa la strada già prevista per le carte che una scansione non ce l'hanno.

E `loading="lazy"`?

Nella griglia le immagini non hanno `src` finché non servono: l'URL sta in `data-src` e ce lo mette un `IntersectionObserver` quando la card si avvicina al viewport. Non è pignoleria: `loading="lazy"` su un `<img>` inserito via `innerHTML` **dentro uno Shadow DOM** non si attiva mai, e con centinaia di carte significherebbe scaricarle tutte insieme. Il rovescio della medaglia è il CSS `img:not([src]) { visibility: hidden }`, che tiene lo spazio senza mostrare l'icona di immagine vuota nell'attesa.

4. Il segnaposto: un SVG inline, non un carattere

Il disegno che prende il posto della foto sta in un modulo solo (`src/ui/segnaposto.js`) e viene usato da griglia, scheda, pannello di aggiunta e carosello. È un **SVG scritto a mano dentro il JS**, e le tre ragioni per cui non è un'emoji né un carattere tipografico sono tutte pratiche:

1. **currentColor** — dentro l'SVG, `stroke="currentColor"` eredita la `color` CSS dell'elemento che lo ospita. Siccome quella `color` è `var(--tipo-colore)`, il segnaposto di un Pokémon Psico è viola e quello di uno Erba è verde, **senza una riga di JavaScript**: la custom property attraversa il confine, l'SVG la raccoglie;
2. **niente font** — un carattere Unicode dipende dai font installati, e su un telefono può diventare un rettangolo vuoto;
3. **scala** — è vettoriale: nitido a 40 px nel carosello come a 140 px nella scheda, con una sola definizione.

L'elemento porta `aria-hidden="true"`: il nome della carta è già scritto accanto, e farlo annunciare due volte è rumore, non accessibilità.

5. Esercizi

1. Nella griglia, l'`<img>` diventa segnaposto quando il caricamento fallisce. Cosa succede oggi se la *stessa* card viene ridisegnata (per esempio cambiando un filtro)? Traccia il percorso da `#disegnaRisultati()` a `#card()` e spiega perché il segnaposto non “resta appiccicato” alla carta.
2. `seImmagineRotta()` accetta la carta e una classe CSS. Prova a immaginare la variante che, invece di un disegno, mostri il **nome** della carta come fa il carosello per i proxy senza scansione: quali dei quattro punti d'uso ci guadagnerebbero e quali no?
3. Perché `alt=""` e l'assenza totale di `alt` non sono equivalenti? Verificalo: apri la griglia, toglì l'attributo da un'immagine con gli strumenti per sviluppatori e rompine l'URL.